Osnove informacionih sistema

OSNOVE INFORMACIONIH SISTEMA

Tutorijal 8

17.12.2024.

Faza dizajna

- Design Specification Document – Dokument Specifikacija dizajna proizvoda detaljan je dokument koji pruža informacije o dizajniranom proizvodu ili procesu.
- Dizajn sistema daje sveobuhvatne smjernice o funkcijama sistema, zahtjevima performansi, bezbjednosnim zahtjevima, karakteristikama platforme i treba da sadrži:
  - Dizajn arhitekture
  - Poslovna pravila
  - Dizajn interfejsa
  - Dizajn baze podataka (pohrana podataka i db pristup)
  - Neki dodatni artefakti / dokumenti proizvedeni zbog aktivnosti u ovoj fazi uključuju:
    - Rječnik podataka
    - Dijagrami programa
    - Dijagrami rasporeda ekrana
    - tabele poslovnih pravila
    - prototip / Proof-of-Concept

Dizajn arhitekture

- Dizajn arhitekture djeluje kao preliminarni plan za razvoj softvera.

- IEEE definira dizajn arhitekture kao "proces definiranja skupa hardverskih i softverskih komponenti i njihovih interfejsa kako bi se uspostavio okvir za razvoj računarskog sistema.„

- Svaki dizajn treba opisati:
  - Set komponenti (npr. Baza podataka, računarski moduli) koji će izvršiti funkciju koju zahtjeva sistem.
  - Set konektora koji će pomoći u koordinaciji, komunikaciji i saradnji između komponenti.
  - Uslovi kako se komponente mogu integrirati u formiranje sistema.
  - Semantički modeli koji pomažu dizajneru da razumije cjelokupne karakteristike sistema.

Dizajn arhitekture

- U softverskom inženjerstvu postoji tijesna veza između nefunkcionalnih zahtjeva (NFR) i softverskih arhitektura (SA).
- Arhitekti softvera moraju stalno da se bave NFR-jevima kao dio njihove odgovornosti SA dizajna. Moraju znati NFR sistem i kako arhitektonske odluke utječu na ispunjavanje NFR-a.
- Nefunkcionalni zahtjevi se mogu podijeliti u četiri kategorije:
  - Rad – prenosivost i kompatibilnost (1)
  - Performanse i skalabilnost (2)
  - Sigurnost (3)
  - Kulturalni i politički zahtjevi (4)

Dizajn arhitekture: NFR – (1) i (2)

- **Prenosivost** definira kako se sistem ili njegov element može pokrenuti u jednom okruženju. Obično uključuje specifikacije hardvera, softvera ili druge upotrebne platforme.
- **Prenosivost** ima i dodatni aspekt koji se zove kompatibilnost. Kompatibilnost definira kako sistem može koegzistirati s drugim sistemom u istom okruženju.
- **Prenosivost** i kompatibilnost utvrđeni su u pogledu operativnih sistema, hardverskih uređaja, pretraživača, softverskih sistema i njihovih verzija.
- **Performanse** definiraju koliko brzo softverski sistem ili njegov određeni dio reagira na radnje određenih korisnika pod određenim radnim opterećenjem.
- **Skalabilnost** ocjenjuje najveće radno opterećenje pod kojim će sistem i dalje ispunjavati zahtjeve za radnim zahtjevima.

Dizajn arhitekture: NFR – (3) i (4)

- **Sigurnost** osigurava da će svi podaci unutar sistema ili njegovog dijela biti zaštićeni od napada zlonamjernog softvera ili neovlaštenog pristupa.

- Najjednostavnije rečeno cilj sigurnosti jeste da spriječi ono što nije dozvoljeno.

- Provođenje sigurnosne politike svakako povećava troškove.
- Ulaganje u sigurnost ne bi trebalo biti veće od vrijednosti onog što se štiti.

- **Kulturalni i politički zahtjevi** se razlikuju od države do države u kojoj se sistem koristi.

- **Kontekst** uključuje lokalne jezike, zakone, valute, kulture, pravopise i druge aspekte. Što više proizvoda je u skladu sa kulturalnim i politčkim zahtjevima to bi trebao imati više uspjeha s određenom ciljnom publikom.

Sigurnost

- Pri računanju rizika se najčešće koristi očekivani godišnji gubitak **ALE (Annualized Loss Expectancy)**
  - Prvo odredimo vrijednost imovine **AV (Asset Value)** (trošak kupovine,instalacije,obuke, direktna dobit ostvarena od imovine)
  - AV množimo sa faktorom izloženosti **EF (Exposure Factor)**
    - **EF** je subjektivni potencijalni procenat gubitka određene imovine ako se realizuje određena opasnost. Opisuje gubitak koji će se dogoditi imovini kao rezultat prijetnje. Ako je vrijednost imovine smanjena u dvije trećine, vrijednost faktora izloženosti iznosi 66%. Ako je sredstvo potpuno izgubljeno, faktor izloženosti je 100%.
  - **Očekivani jednokratni gubitak** se računa kao **SLE (single loss expectancy)** = AV * EF
  - Na kraju SLE pomnožimo sa **godišnjom učestalosti događaja ARO (annual rate of occurrence)** i dobijemo **očekivani godišnji gubitak ALE = SLE × ARO**

Primjer 1

Izračunati ALE sa godišnjom učestalosti događaja 1 i 3, ukoliko je vrijednost imovine procijenjena na $100,000, a faktor izloženosti na 25%.

- Pri računanju rizika se najčešće koristi očekivani godišnji gubitak ALE
  - Prvo odredimo vrijednost imovine AV (Asset Value) (trošak kupovine,instalacije,obuke, direktna dobit ostvarena od imovine)
  - AV množimo sa faktorom izloženosti EF (Exposure Factor)
    - EF je subjektivni potencijalni procenat gubitka određene imovine ako se realizuje određena opasnost. Opisuje gubitak koji će se dogoditi imovini kao rezultat prijetnje. Ako je vrijednost imovine smanjena u dvije trećine, vrijednost faktora izloženosti iznosi 66%. Ako je sredstvo potpuno izgubljeno, faktor izloženosti je 100%.
  - Očekivani jednokratni gubitak se računa kao SLE (single loss expectancy) = AV * EF
  - Na kraju SLE pomnožimo sa godišnjom učestalosti događaja ARO (annual rate of occurrence) i dobijemo očekivani godišnji gubitak ALE = SLE × ARO

Oznake:

- ALE (Annualized Loss Expectancy /očekivani godišnji gubitak)
- AV (Asset Value/ vrijednost imovine)
- EF (Exposure Factor/ faktorom izloženosti)
- SLE (Single Loss Expectancy/ očekivani jednokratni gubitak)
- ARO (Annual Rate of Occurrence/ godišnjom učestalosti događaja)

Primjer 1

- Asset value:$100,000,
- Exposure Factor (EF) :25%.
- Single loss expectancy (SLE) =25% * $100,000 = $25,000.

1. ARO = 1
   - Annualized loss expectancy(ALE) = 1 * $25,000= $25,000.

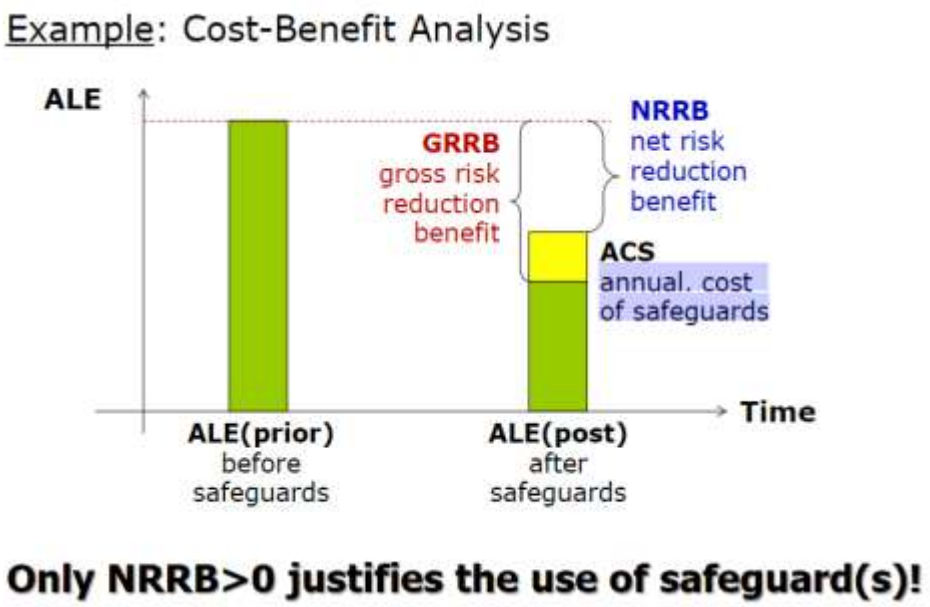
2. ARO=3
   - ALE = 3 * $25,000 = $75,000

Oznake:
- ALE (Annualized Loss Expectancy /očekivani godišnji gubitak)
- AV (Asset Value/ vrijednost imovine)
- EF (Exposure Factor/ faktorom izloženosti)
- SLE (Single Loss Expectancy/ očekivani jednokratni gubitak)
- ARO (Annual Rate of Occurrence/ godišnjom učestalosti događaja)

Sigurnost

- **Vrijednost mjere zaštite = ALE (bez mjere) – Trošak (provođenja mjere) – ALE (sa mjerom)**
- Pri izračunavanju rizika zbog virusa očekuje se godišnji gubitak (ALE) od 145.000 dolara. Troškovi antivirusnog programa procjenjuju se na 24.000 dolara godišnje, a smanjit će ALE na 65.000 dolara.
- Da li je to isplativa protivmjera? Zašto ili zašto ne?
- ALE (prior) = $ 145k
- ALE (post) = $ 65 k
- ACS = $ 24 k

- NRRB = ALE (prethodno) - ALE (post) - ACS =
- = $ 145 k - $ 65 k - $ 24 k =
- = $ 56 k, tako da postoje + troškovi korištenja ovog rješenja
- *NRRB - NET risk reduction benefit (sačuvani novac)
- ** Scenario najboljeg slučaja bi bilo kada bi ALE sa mjerom =0
- ***ACS = annual. cost of safeguards



Primjer 2

Izračunati ALE Web stranice zbog DDoS napada. Procijenjena vrijednost web stranice je 1.000.000 USD. DDoS na sajtu bi rezultirao gubitkom od 10% vrijednosti stranice. Napad se dogodi jednom u 2 godine.

Primjer 2

- AV = 1.000.000 USD
- EF=0,1
- SLE : AV * EF = $ 100,000
- ARO=0.5
- ALE = ARO * SLE = $ 50 000

Primjer 3

- Koji je najveći iznos od ponuđenih koji biste preporučili korporaciji da godišnje investira u kontramjeru za zaštitu svoje imovine u vrijednosti od $ 1 milion od potencijalne prijetnje koja ima stopu pojavljivanja jednom svakih pet godina i faktor izloženosti 10%.
- A) 100 000
- B) 20 000
- C) 200 000
- D) 40 000

Primjer 3

- ALE (prior) = AV*EF*ARO = $\$10^6*0.1*0.2 = \$20,000$
- ALE (post) = $0 (best case scenario - safeguard 100% eff.)
- ACS = ?
- Za NRRB ≥0, do $20,000 je prihvatljivo.

Sigurnost

Da li je vrijednija oprema ili su vrijedniji podaci?

Sigurnost

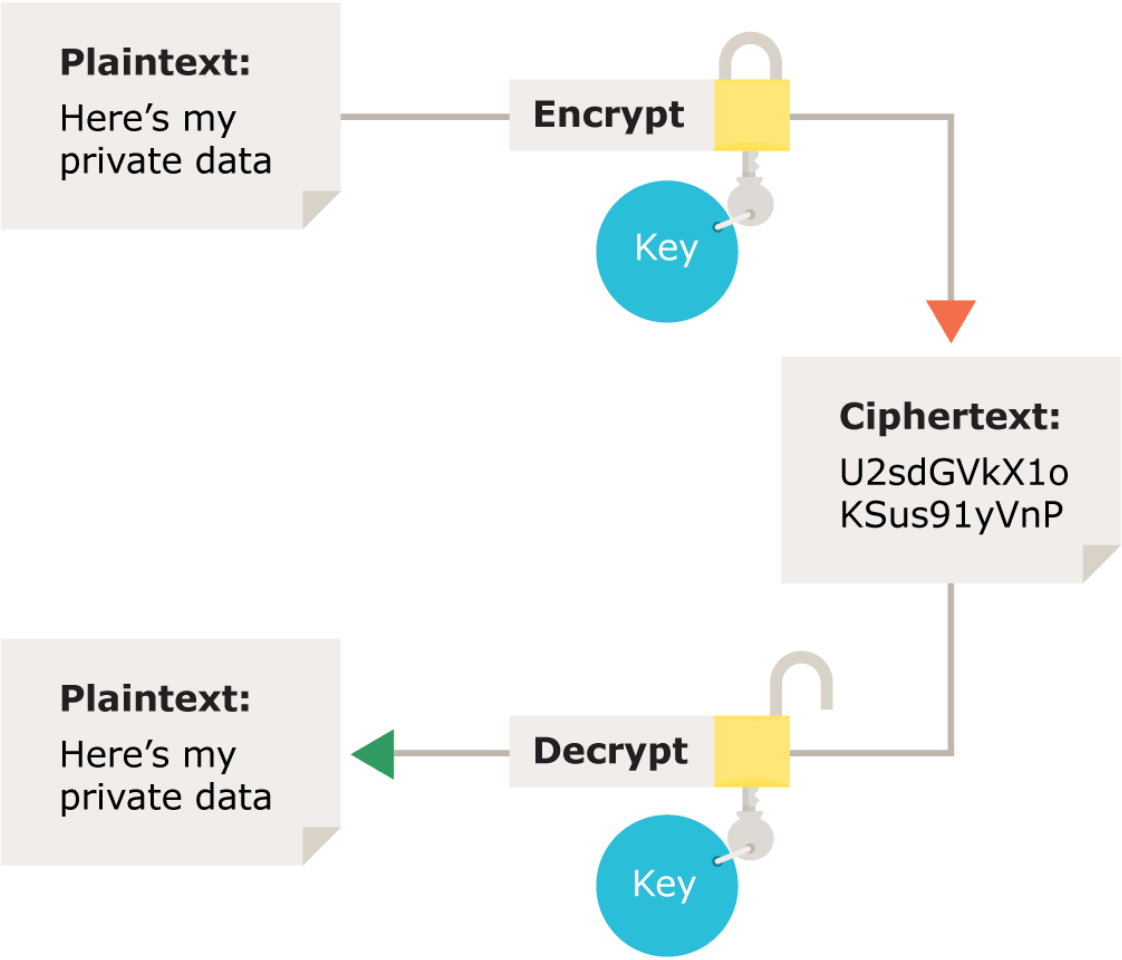
Da li je vrijednija oprema ili su vrijedniji podaci?

PODACI

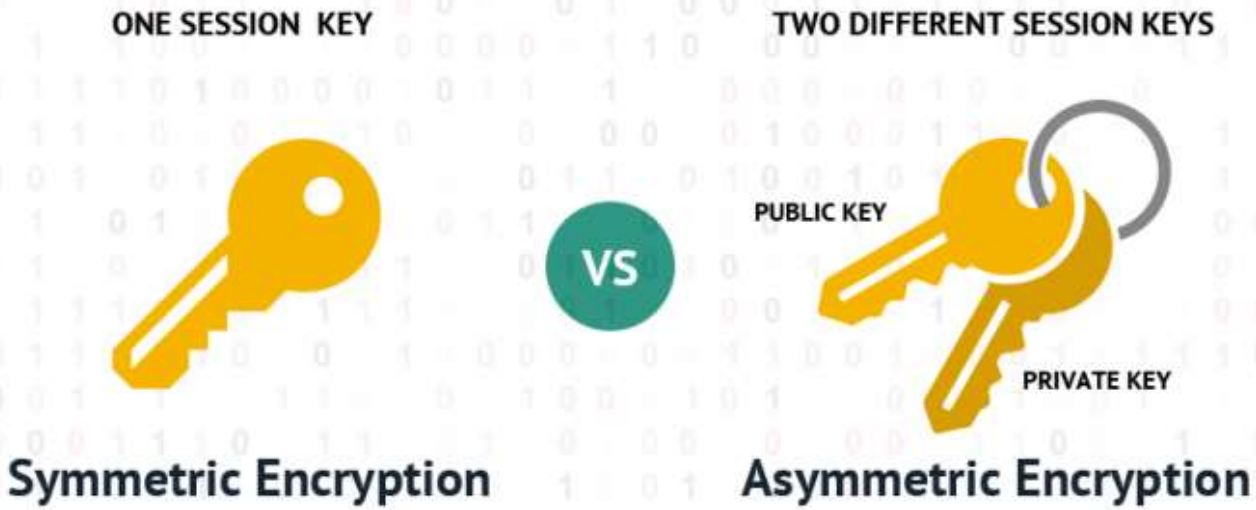
Sigurnost

- Kontrola pristupa:
  - Autentifikacija – potvrđivanje identiteta
    - Nešto što znaš (password)
    - Nešto što posjeduješ (smart card)
    - Nešto što jesi (otisak prsta ili druge biometrijske metode)
  - Autorizacija - provjera ovlaštenja subjekata nad objektima sistema
- Enkripcija:
  - Simetrična i asimetrična
  - Digitalni potpis

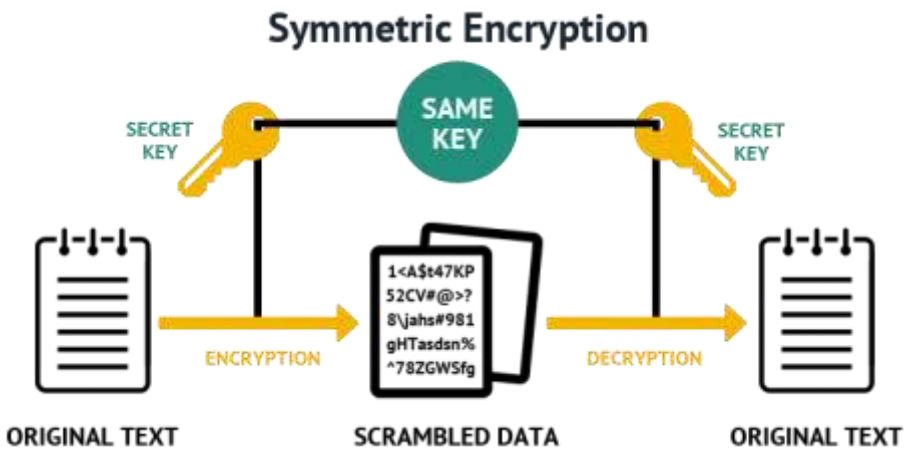
Sigurnost

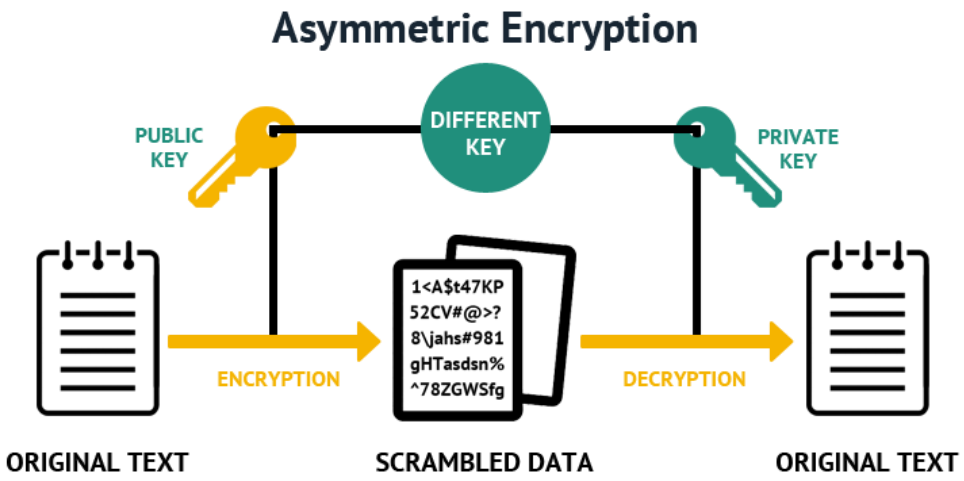

Plaintext:
Here's my private data
Encrypt
Key
Ciphertext:
U2sdGVkX1o KSus91yVnP
Decrypt
Key
Plaintext:
Here's my private data

Sigurnost

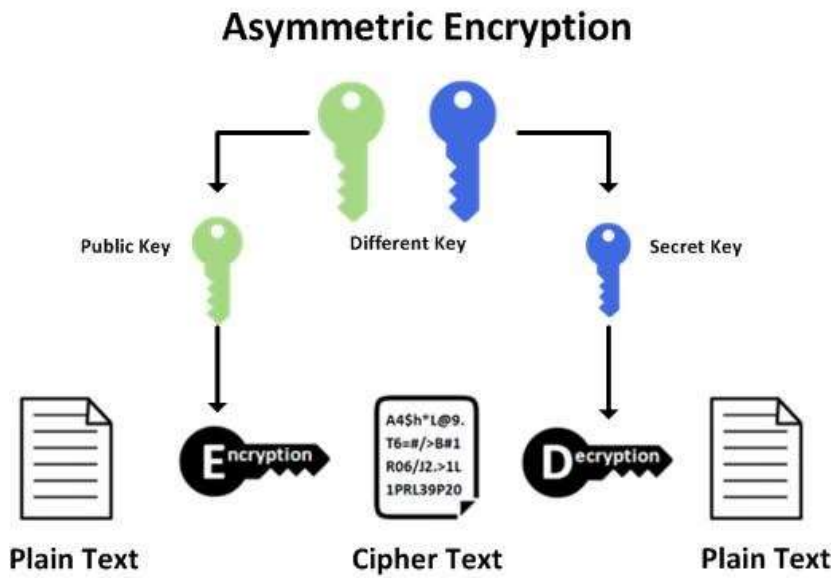

ONE SESSION KEY
TWO DIFFERENT SESSION KEYS
VS
PUBLIC KEY
PRIVATE KEY
Symmetric Encryption
Asymmetric Encryption

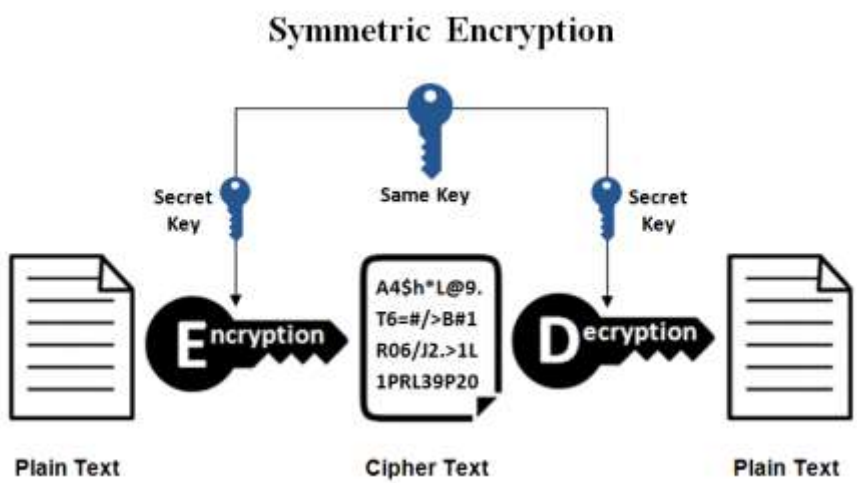
Sigurnost


Symmetric Encryption
SECRET KEY
SAME KEY
SECRET KEY
ENCRYPTION
1<A$t47KP
52CV#@>?
8\jahs#981
gHTasdsn%
^78ZGWSfg
DECRYPTION
ORIGINAL TEXT
SCRAMBLED DATA
ORIGINAL TEXT

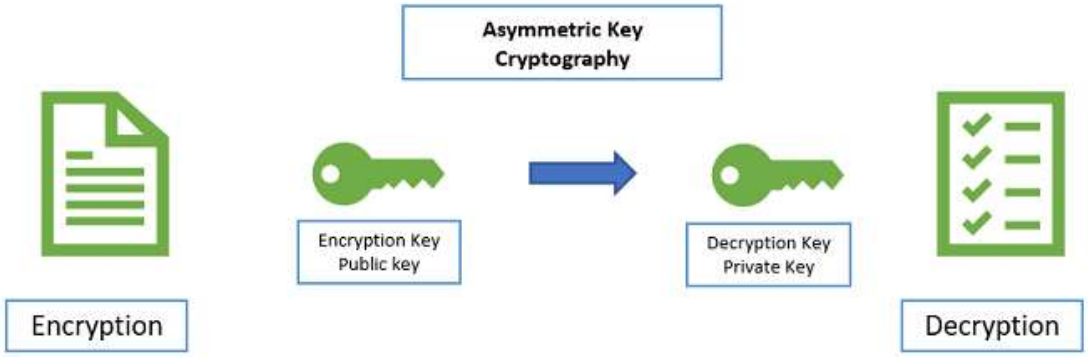

Asymmetric Encryption
PUBLIC KEY
DIFFERENT KEY
PRIVATE KEY
ENCRYPTION
1<A$t47KP
52CV#@>?
8\jahs#981
gHTasdsn%
^78ZGWSfg
DECRYPTION
ORIGINAL TEXT
SCRAMBLED DATA
ORIGINAL TEXT

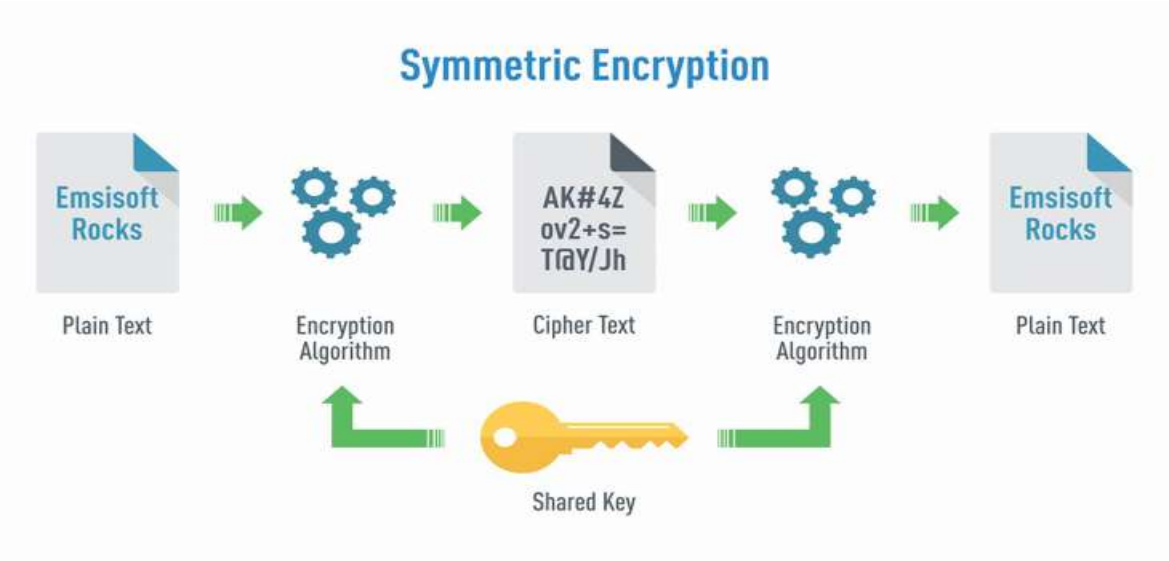
Sigurnost


Asymmetric Encryption
Public Key
Different Key
Secret Key
Encryption
A4$h*L@9.
T6=#/>B#1
R06/J2.>1L
1PRL39P20
Decryption
Plain Text
Cipher Text
Plain Text


Symmetric Encryption
Secret Key
Same Key
Secret Key
Encryption
A4$h*L@9.
T6=#/>B#1
R06/J2.>1L
1PRL39P20
Decryption
Plain Text
Cipher Text
Plain Text

Sigurnost


Asymmetric Key Cryptography
Encryption Key Public key
Decryption Key Private Key
Encryption
Decryption


Symmetric Encryption
Emsisoft Rocks
AK#4Z ov2+s= T@Y/Jh
Emsisoft Rocks
Plain Text
Encryption Algorithm
Cipher Text
Encryption Algorithm
Plain Text
Shared Key

Testiranje

https://www.javainuse.com/aesgenerator

https://www.devglan.com/online-tools/rsa-encryption-decryption

Digitalni potpis

- **Digitalni potpis** je matematička tehnika koja se koristi za provjeru autentičnosti i integriteta poruke, softvera ili digitalnog dokumenta.
- To je digitalni ekvivalent rukom pisanog potpisa ili pečata, ali nudi daleko veću sigurnost.
- **Digitalni potpis** je namijenjen rješavanju problema neovlaštenog pristupa i lažnog predstavljanja u digitalnoj komunikaciji.
- **Digitalni potpisi** mogu pružiti dokaz o porijeklu, identitetu i statusu elektronskih dokumenata, transakcija ili digitalnih poruka. Potpisnici ih također mogu koristiti za potvrdu informiranog pristanka.

- **Autentičnost** Potpisnik se potvrđuje kao potpisnik.
- **Integritet** Sadržaj nije mijenjan i njime se nije neovlašteno manipuliralo otkako je digitalno potpisan.
- **Zajamčeno priznavanje** Svim stranama dokazuje izvor potpisanog sadržaja. Nepriznavanje znači da potpisnik ne priznaje nikakvu vezu s potpisanim sadržajem.

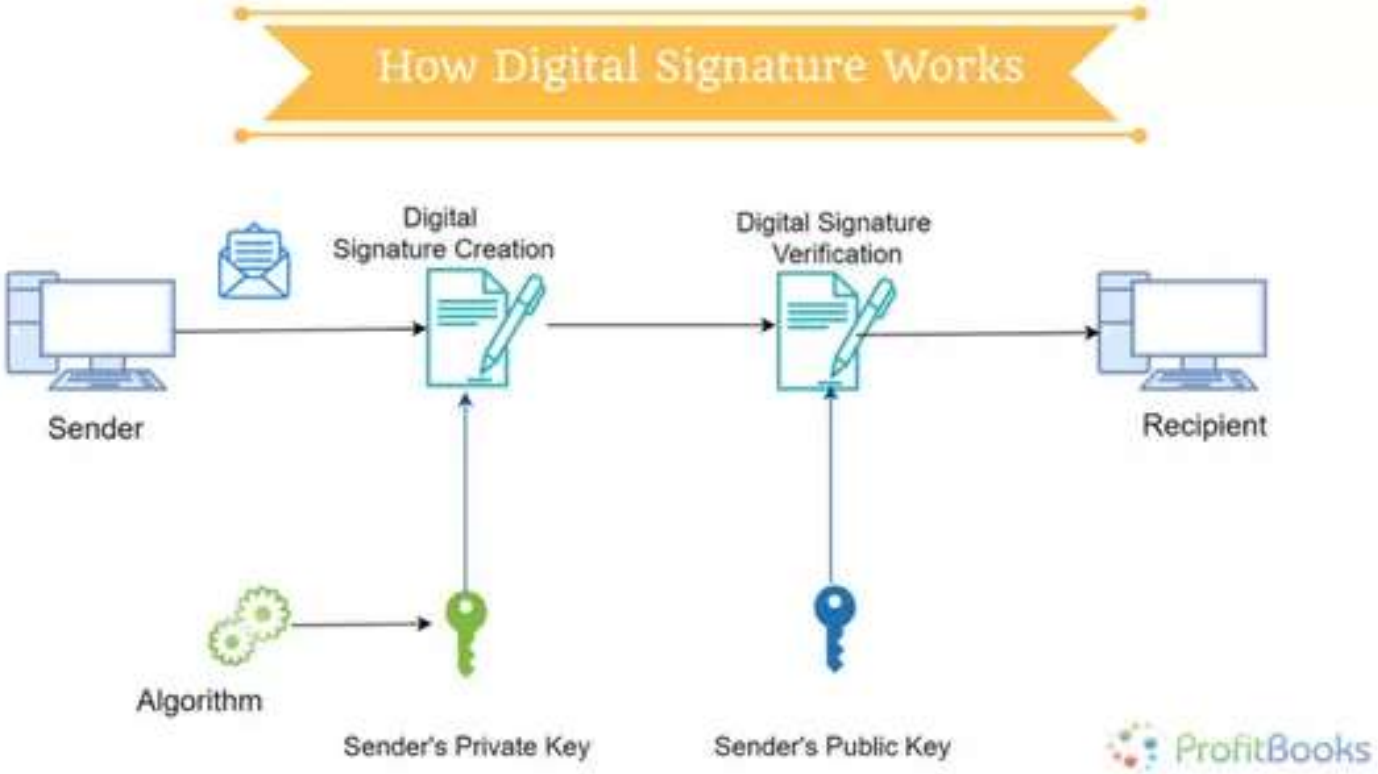
Ustanova za izdavanje certifikata (Certificate Authority, CA)

Ustanova za izdavanje certifikata subjekt je sličan javnom bilježniku. Ona izdaje digitalne certifikate, potpisuje certifikate da bi se potvrdila njihova valjanost te bilježi opozvane certifikate i certifikate čija je valjanost istekla.

Potpisivanje certifikata

Da biste stvorili digitalni potpis, morate imati certifikat za potpisivanje, što dokazuje identitet. Kada šaljete digitalno potpisanu makronaredbu ili dokument, šaljete i certifikat i javni ključ. Certifikate izdaje ustanova za izdavanje certifikata i kao vozačka dozvola može se opozvati. Potvrda je obično valjana godinu dana, nakon čega potpisnik mora obnoviti ili nabaviti novu potpisnu potvrdu da bi uspostavio identitet.

Digitalni potpis



File server arhitektura

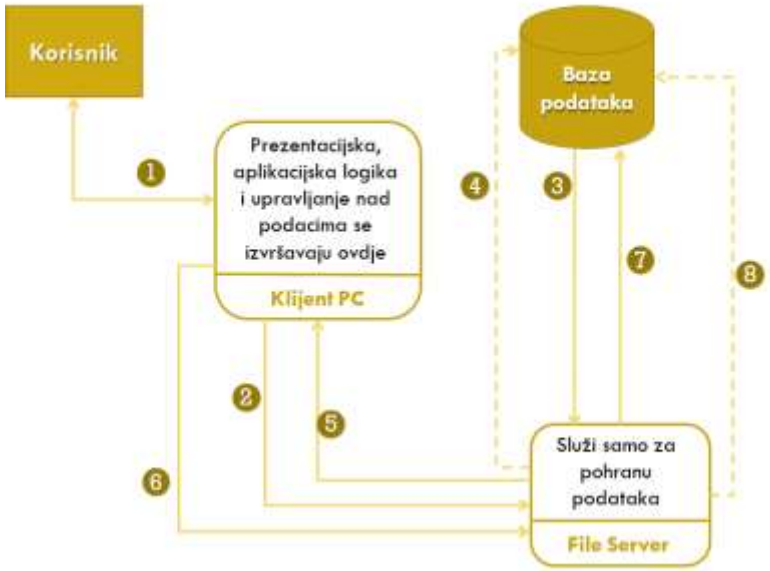
- koristi se za razmjenu poruka i informacija preko mreže radi pronalaženja traženih podataka.

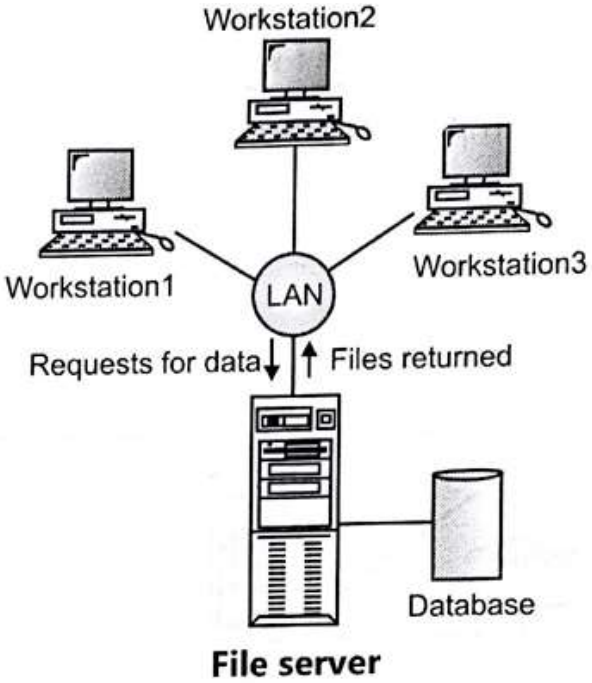
- Klijent prosljeđuje zahtjev za kreiranjem, čitanjem, izmjenom ili brisanjem jednog ili nekoliko zapisa putem mreže na file server

- Kao odgovor na zahtjev, file server vraća čitavu tabelu
- Klijent vraća čitavu tabelu, sa izmijenjenim zapisima preko prezentacijske logike korisniku

File server arhitektura

- **Nedostaci ovog pristupa su sljedeći:**
  - Ogromna količina nepotrebnih podataka se razmjenjuje između klijenta i servera.
  - Klijentski računari moraju biti snažni (tzv. eng. *fat client*).
  - Integritet baze podataka može lako biti narušen.



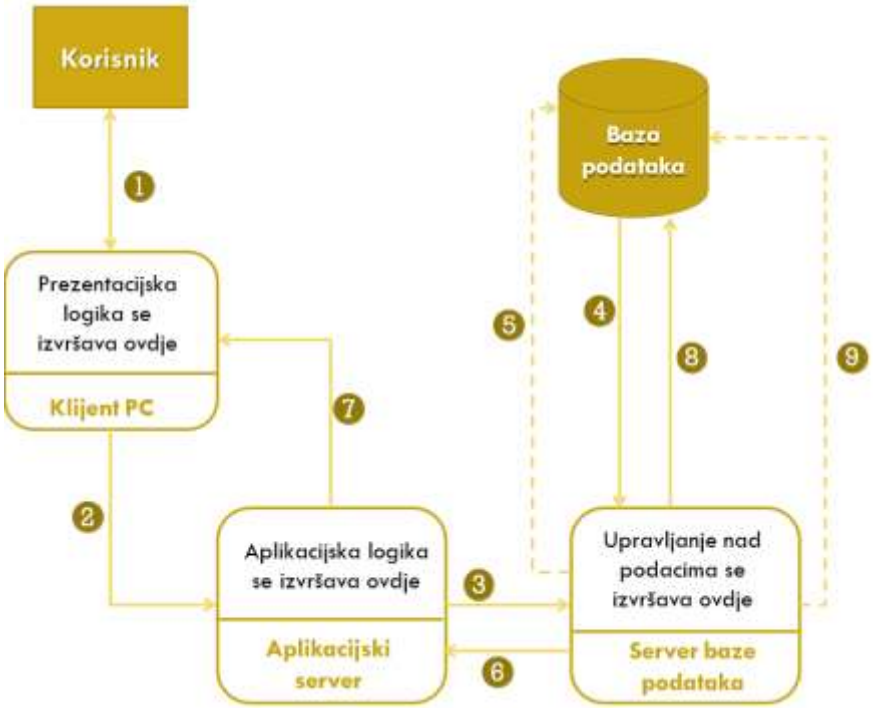


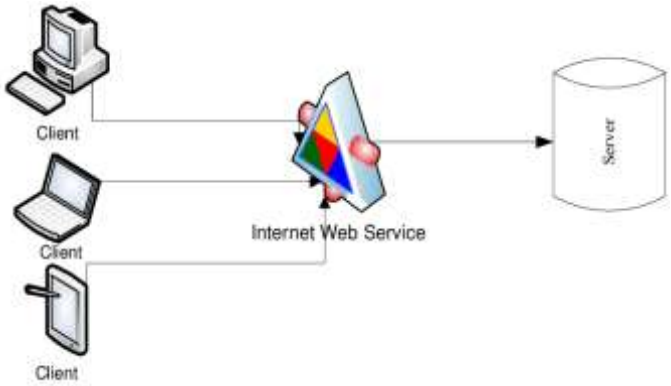
Klijent server arhitektura

- model u kojem server, isporučuje i upravlja većinom resursa i usluga koje klijent treba konzumirati. Ova vrsta arhitekture ima jedan ili više računara klijenta spojenih na središnji server putem mrežne ili internetske veze. Ovaj sistem dijeli računske resurse.

- Fat client:
  - sadrži slojeve prezentacije i aplikacijske logike
  - zahtijeva veću snagu obrade i veći mrežni saobraćaj
- Thin client:
  - sadrži samo sloj prezentacije
  - manja snaga obrade
  - manja količina prenošenih podataka

Klijent server arhitektura



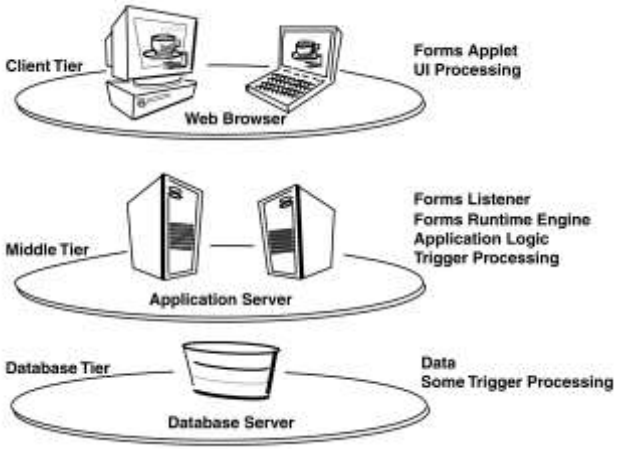


Web bazirana arhitektura

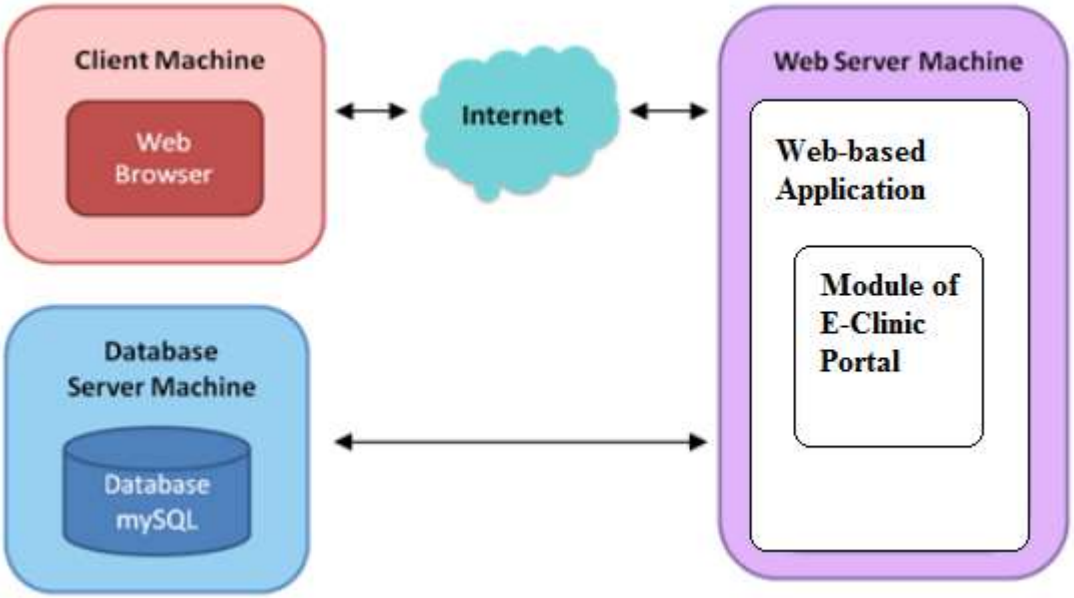
- Web-bazirana arhitektura je višeslojna arhitektura u kojoj su slojevi prezentacije i prezentacijske logike implementirani na strani klijenta (web browser) uz korištenje sadržaja, koji je preuzet od web servera. Sloj prezentacijske logike je vezan za sloj aplikacijske logike koji se pokreće na aplikacijskom serveru, koji se veže na server baze podataka.

- Možda najveći potencijal web tehnologije je mogućnost prebacivanja tradicionalnih informacionih sistema na intranet.

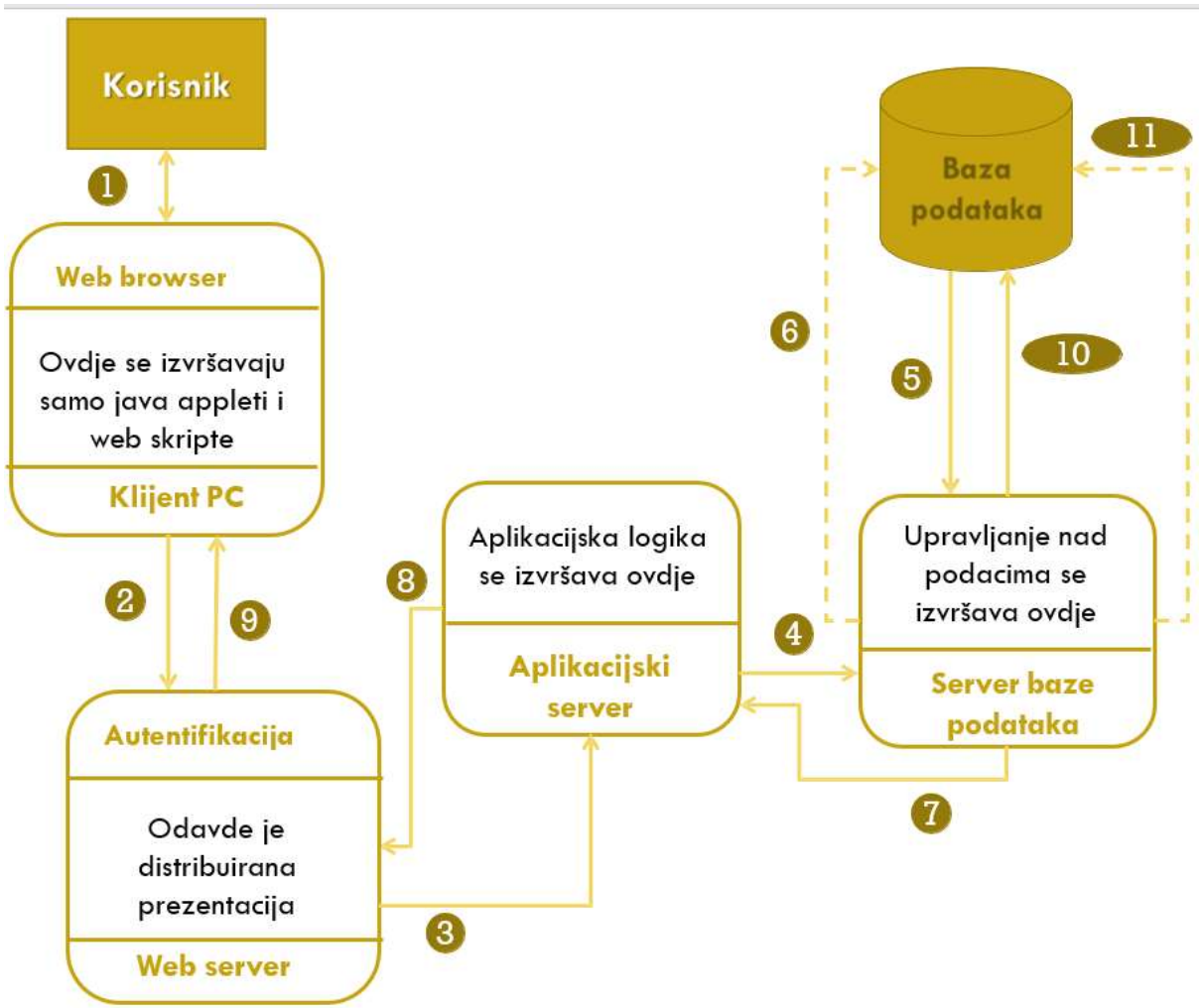
- Intranet je privatna računarska mreža koja koristi internet protokol i mrežnu povezanost da sa sigurnošću dijeli dio informacija jedne organizacije ili operacije između njenih zaposlenih.



Web bazirana arhitektura



Web bazirana arhitektura



Specifikacija hardvera

Specifikacije računarskog hardvera su tehnički opisi komponenti i mogućnosti računara.

- Brzina procesora, model i proizvođač. Brzina procesora obično je prikazana u gigahercima (GHz).
- RAM memorija (RAM), obično je naznačena u gigabajtima (GB).
- Hard disk – količina informacija na istom u GB, TB
- Ostale specifikacije mogu uključivati mrežne (ethernet ili wi-fi) adaptere ili audio i video mogućnosti

Specifikacija hardvera

Specifikacija računarskog softvera predstavlja navođenje i opis istog potrebnog za rad informacionog sistema.

- Operativni sistem
- Poseban softver

Specifikacija hardvera - primjer

Primjer specifikacije

	Standardni klijent	Standardni web server	Standardni aplikacijski server	Standardni server baze podataka
Operating System	• Windows • Mozilla	• Linux	• Linux	• Linux
Special Software	• Real Audio • Adobe Acrobat Reader	• Apache	• Java	• Oracle
Hardware	• 40-gig disk drive • Pentium • 17-inch Monitor	• 80-gig disk drive • Pentium	• 80-gig disk drive • Pentium	• 200-gig disk drive • RAID • Quad Pentium
Network	• Always-on Broadband preferred • Dial-up at 56Kbps possible with some performance loss	• Dual 100 Mbps Ethernet	• Dual 100 Mbps Ethernet	• Dual 100 Mbps Ethernet

Specifikacija hardvera - primjer

	Klijent	Web Server	Aplikacijski Server	Server baze podataka
Operativni sistem	-Windows -Mozilla -Chrome -Torch	Linux	Linux	Linux
Posebni softver	MS Office	Apache	Java	Oracle
Hardver	-Intel Core i3-2370 -22-inčni monitor -500 GB hard disk	-750 GB disk drive -Dual-core Xeon	-100 GB hard disk -Quad-core Xeon	-1 TB hard disk -RAID
Mreža	-Širokopojasna mreža -Dial-up 56 Kbps	Dual 100 Mbps Ethernet	Dual 100 Mbps Ethernet	Dual 100 Mbps Ethernet

Tabela 24.: Specifikacija hardvera i softvera

Dizajn baze podataka

Šta je dobar dizajn baze podataka?

- Razdvaja podatke u tabele zasnovane na subjektu da bi se smanjili redundantni podaci.
- Obezbjeđuje pristup informacijama koje su potrebne za kreiranje određenih informacija.
- Pomaže podršku i osigurava tačnost i integritet podataka.
- Prilagođava potrebe obrade podataka, skalabilnost

Dizajn baze podataka

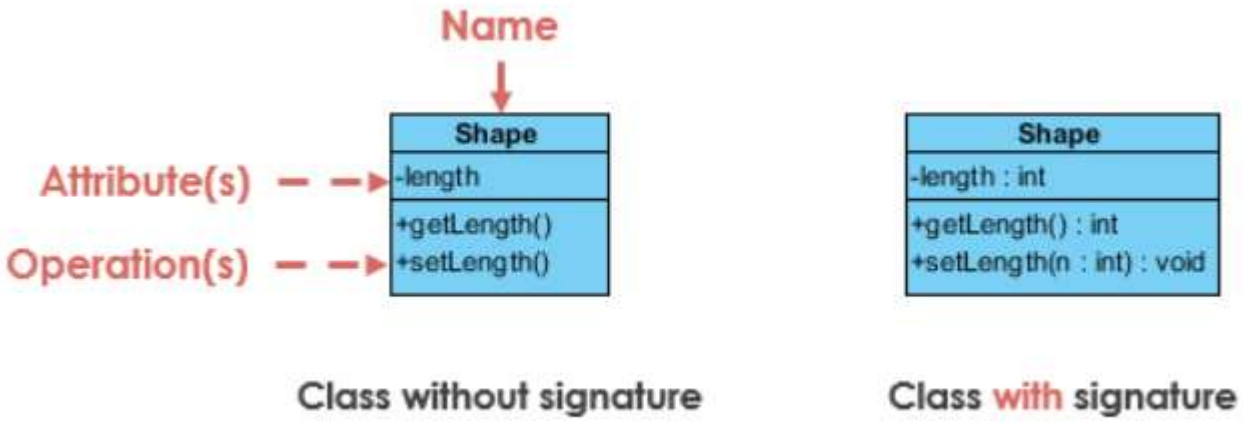
- Atribute implementirati kao polja te za svako naznačiti:
  - Tip podataka
  - Default vrijednost
  - Domena (dopuštene vrijednosti)
  - Veličina polja
  - Null ili not null
- Dodavanje primarnih ključeva
- Dodavanje stranih ključeva

Dijagram klasa

- Dijagram klasa je statički dijagram.
- Predstavlja statički prikaz aplikacije.
- Dijagram klasa se ne koristi samo za vizualizaciju, opisivanje i dokumentovanje različitih aspekata sistema, već i za konstruisanje izvršnog koda softverske aplikacije.

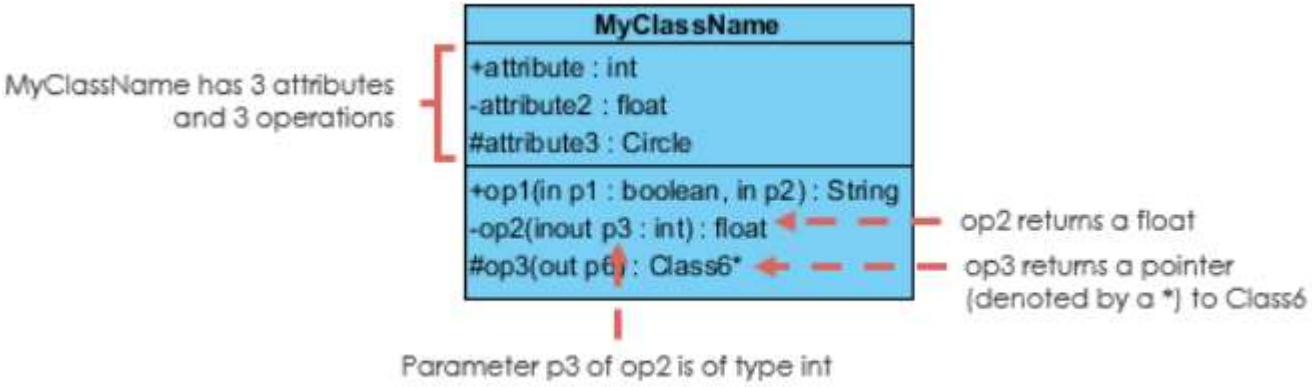
Dijagram klasa

Klasa predstavlja koncept koji obuhvata stanja (atributi) i ponašanja (metode). Svaki atribut ima tip. Svaka metoda ima “potpis”. Naziv klase je jedina obavezna informacija.



Dijagram klasa

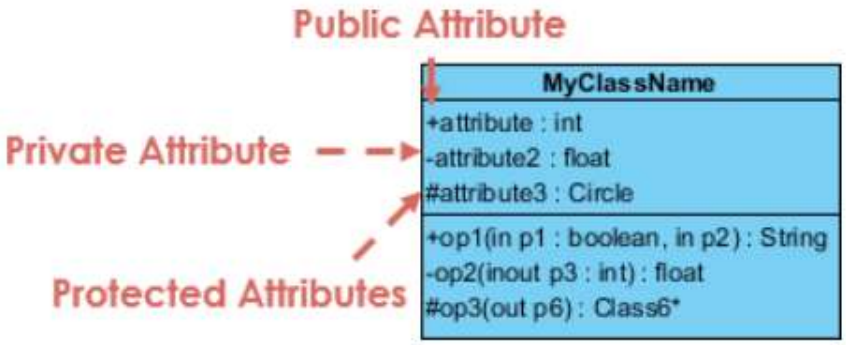
- Naziv klase: Ime klase pojavljuje se na prvoj particiji.
- Atributi klase: Atributi su prikazani na drugoj particiji.Tip atributa prikazan je iza dvotačke
- Metode: Prikazane na trećoj particiji. One su usluge koje klasa pruža.
- Povratni tip metode prikazan je iza dvotačke na kraju potpisa metode. Povratni tip parametra metode prikazuje se iza dvotačke koja slijedi nakon naziva parametra.



Dijagram klasa

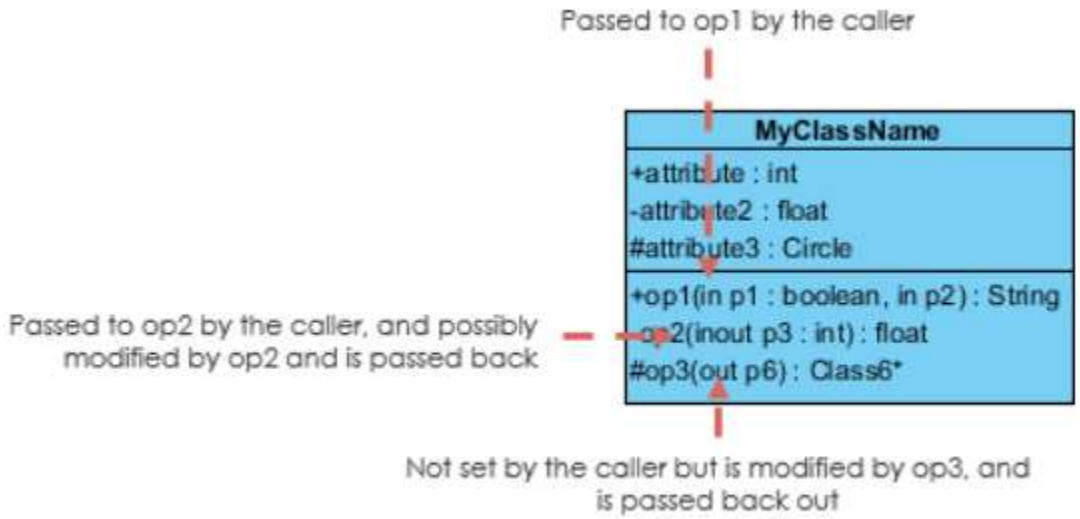
Vidljivost klasa
Simboli +, - i # ispred imena atributa i metoda u klasi označavaju vidljivost atributa i metoda.

- + označava javne(public) atribute ili operacije
- - označava privatne (private)atribute ili operacije
- # označava zaštićene(protected) atribute ili operacije
- ~ označava Package



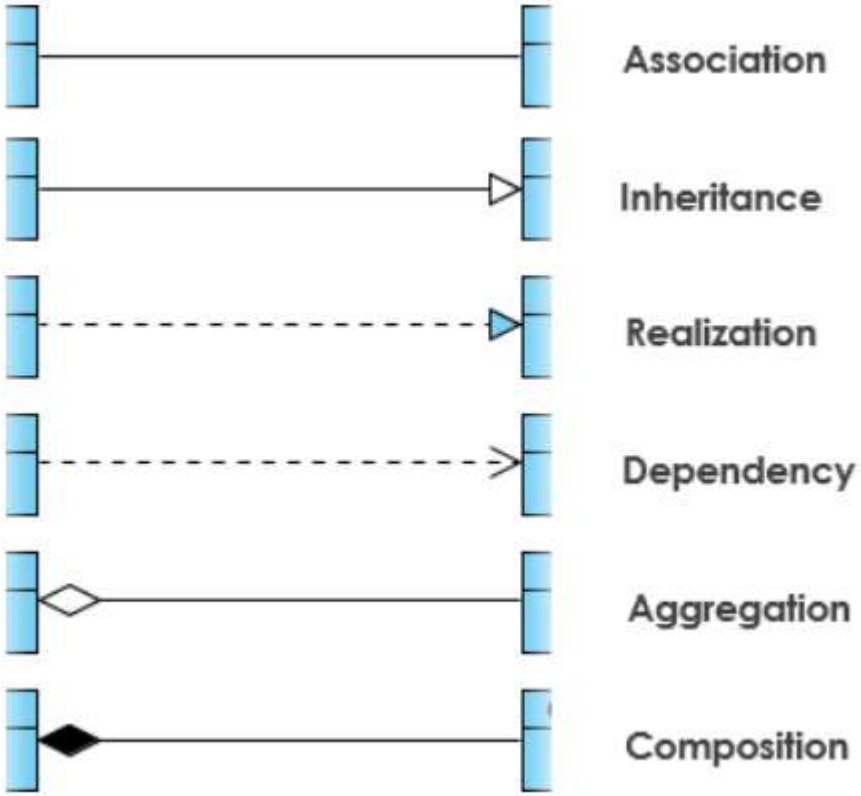
Dijagram klasa

- Usmjerenost parametra
- Svaki parametar u operaciji (metodi) može se označiti kao in, out ili inout te se tako specificira njegov smjer s obzirom na pozivatelja. Ta je usmjerenost prikazana prije naziva parametra.



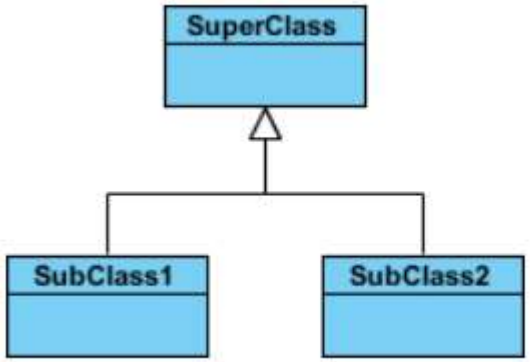
Dijagram klasa

Odnosi između klasa

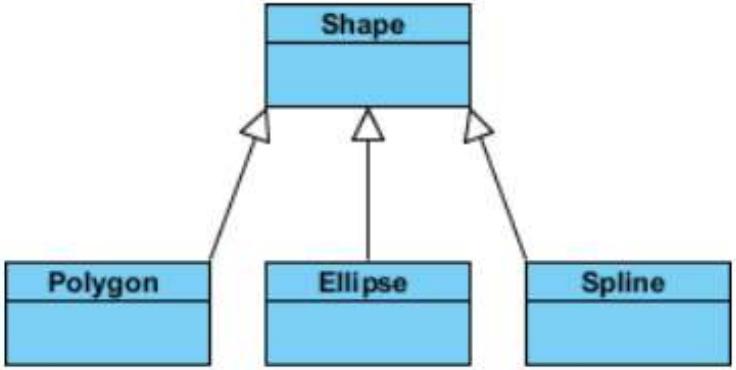


Nasljeđivanje (ili generalizacija)

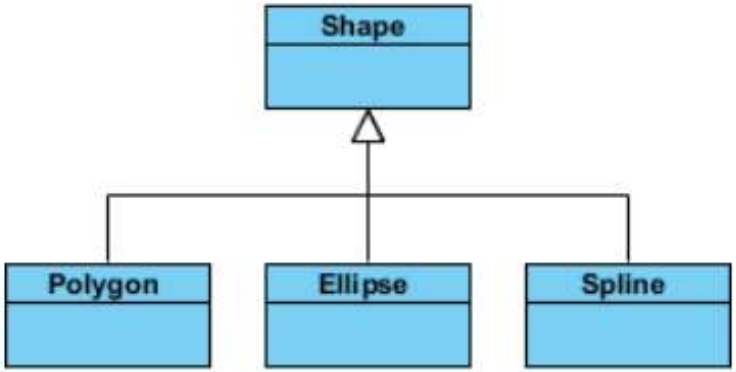
- Nasljeđivanje, odnosno generalizacija, se koristi da pokaže da je jedna klasa specijalizacija druge. Relacije nasljeđivanja su prikazane hijerarhijski, sa baznom klasom na vrhu, i izvedenim klasama ispod njih.
- Odnos je prikazan kao puna crta sa šupljim vrhom strelice koja pokazuje od podređenog elementa do nadređenog elementa.



Nasljeđivanje (ili generalizacija) - primjer



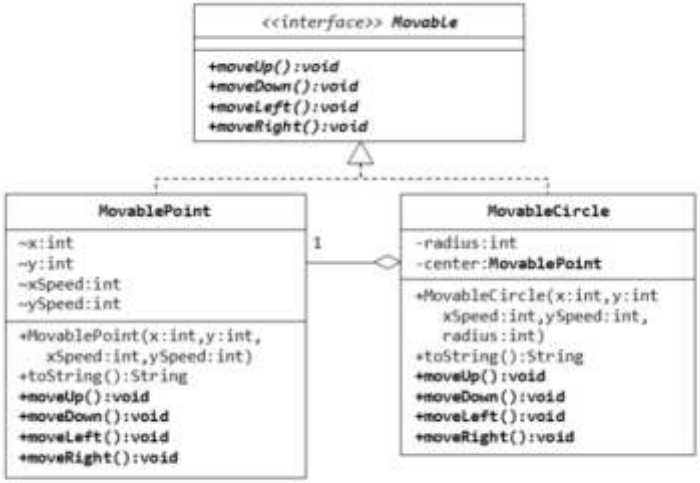
Style 1: Separate target

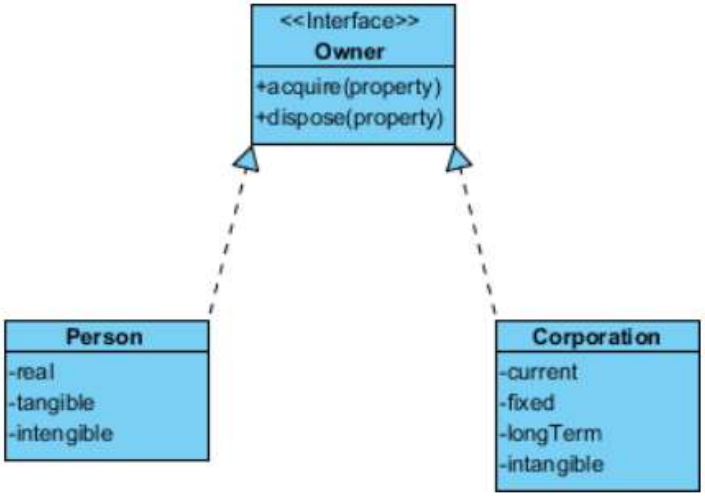


Style 2: Shared target

Realizacija

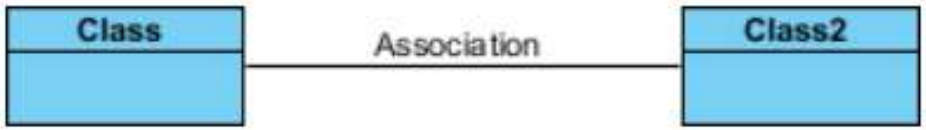
Realizacija predstavlja implementaciju interfejsa od strane neke klase.





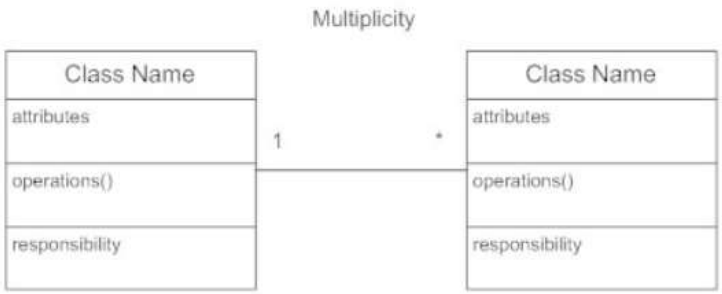
Asocijacija

Ako dvije klase u modelu trebaju međusobno komunicirati, mora postojati veza između njih, a to može biti predstavljeno asocijacijom (konektorom).



Višestrukost (kardinalnost) oznake

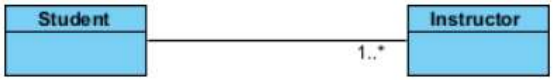
Višestrukosti postavite blizu krajeva asocijacije. Ovi simboli označavaju broj primjeraka jedne klase povezanih s jednim primjerkom druge klase. Primjerice, jedna će kompanija imati jednog ili više zaposlenika, ali svaki zaposlenik radi za samo jednu kompaniju.



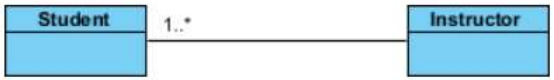
Indicator	Meaning
0..1	Zero or one
1	One only
0..*	0 or more
1..* | *	1 or more
n	Only n (where n > 1)
0..n	Zero to n (where n >1)
1..n	One to n (where n > 1)

Višestrukost (kardinalnost) oznake

A single student can associate with multiple teachers:



The example indicates that every Instructor has one or more Students:

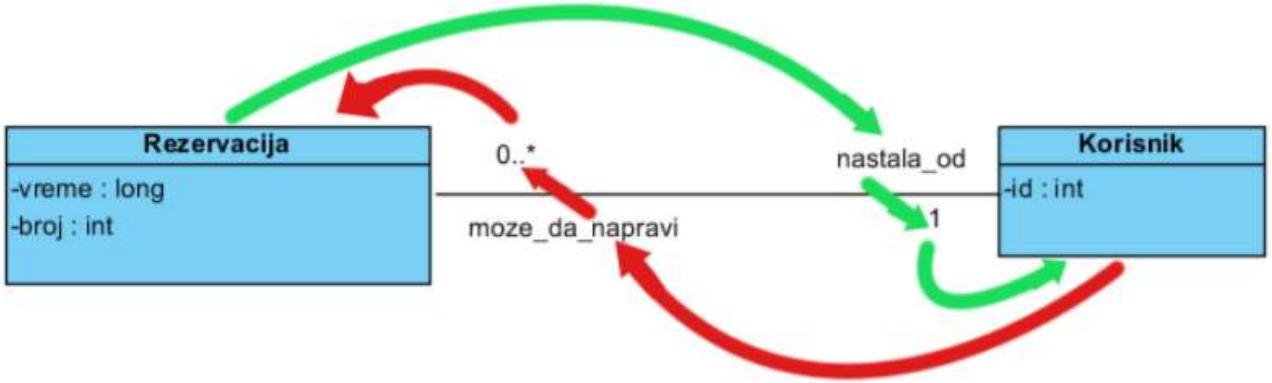


We can also indicate the behavior of an object in an association (i.e., the role of an object) using role names.

Bidirekciona asocijacija

Bidirekciona asocijacija

U slučaju da obe klase *znaju jedna za drugu* i vezu među njima, onda one imaju bidirekcionu asocijaciju.



Dati primer bidirekcione asocijacije se čita:

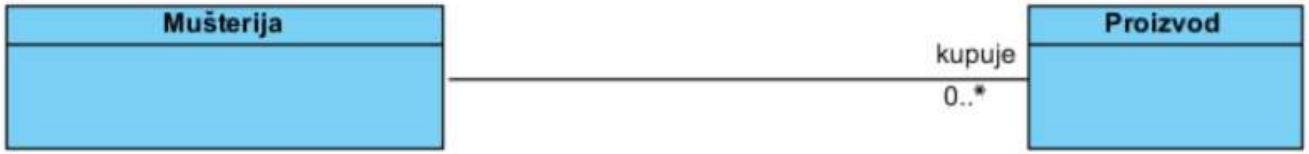
Rezervacija je nastala od tačno jednog korisnika.

Korisnik može da napravi 0 ili više rezervacija.

Unidirekciona asocijacija

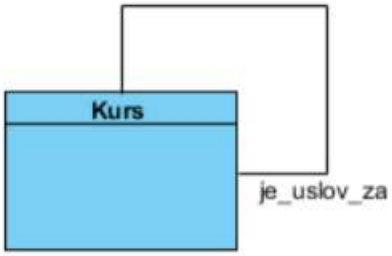
Unidirekciona asocijacija

Asocijacija može da bude i unidirekciona, gde jedna klasa zna za drugu klasu i odnos među njima, ali ***druga klasa ne zna***.



Refleksivna asocijacija

Refleksivna asocijacija

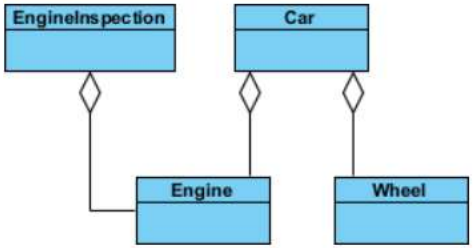


Refleksivna asocijacija se koristi kada je jedna instanca klase u vezi sa drugom instancom *te iste* klase.

Na primer, neki kurs je uslov za upisivanje drugog kursa.

Agregacija

- Agregacija podrazumijeva odnos u kojem dijete može postojati neovisno od roditelja.
- Primjer: Razred (roditelj) i učenik (dijete). Izbrišite razred i učenici i dalje postoje.
- Odnos se prikazuje kao puna crta s neispunjenim dijamantom na kraju asocijacije, koji je povezan s klasom koja predstavlja agregat (klasa cjeline).

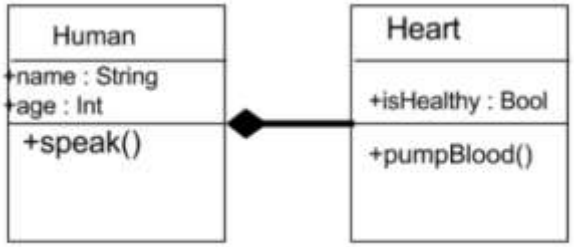


Dakle: motor je dio automobila, ali i bez automobila postojanje motora ima smisla. Asocijacija se razlikuje od agregacije jedino po tome što ne podrazumijeva da jedna klasa sadrži drugu

Kompozicija

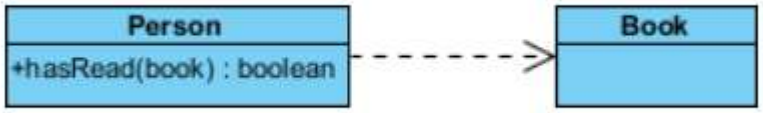
- Kompozicija podrazumijeva odnos u kojem dijete ne može postojati neovisno od roditelja. Primjer: Kuća (roditelj) i Soba (dijete). Sobe ne postoje zasebno od kuće.
- Ne možete srušiti kuću, a da sobe ostanu netaknute.
- Kompozicija se predstavlja popunjenim simbolom dijamanta uz klasu cjeline.

Kuća mora da sadrži barem jednu sobu. Soba može da se nalazi samo u jednoj kući, i nema smisla da postoji bez kuće. Ako se uništi instanca klase kuća onda se uništavaju i instance klase soba unutar nje.

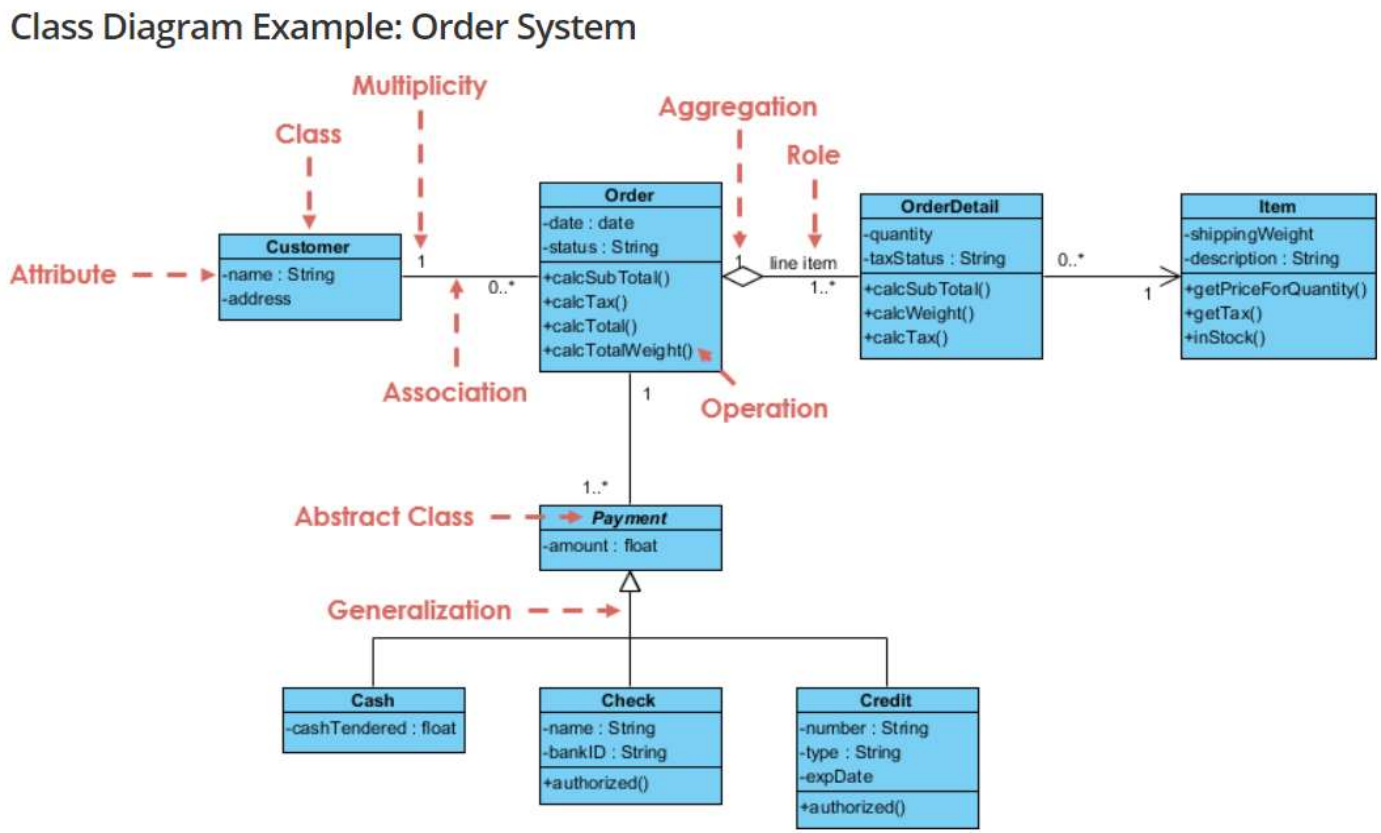


Ovisnost(Dependency)

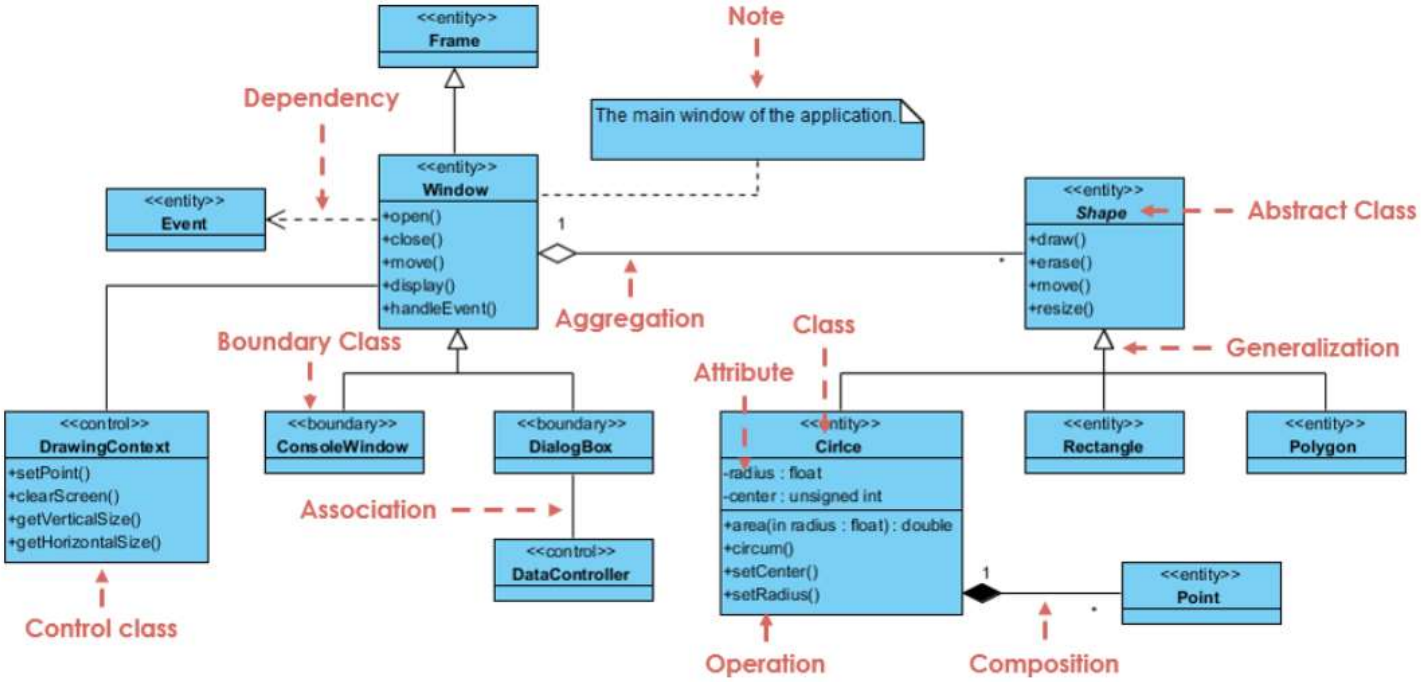
- U UML-u odnos odnosa ovisnosti je odnos u kojem jedan element, klijent, koristi ili ovisi o drugom elementu, dobavljaču.
- Postoji između dvije klase ako promjene definicije jedne mogu uzrokovati promjene druge (ali ne i obrnuto).
- Klasa Person možda ima metodu hasRead s parametrom Book koji vraća true ako je osoba pročitala knjigu (možda provjerom neke baze podataka).



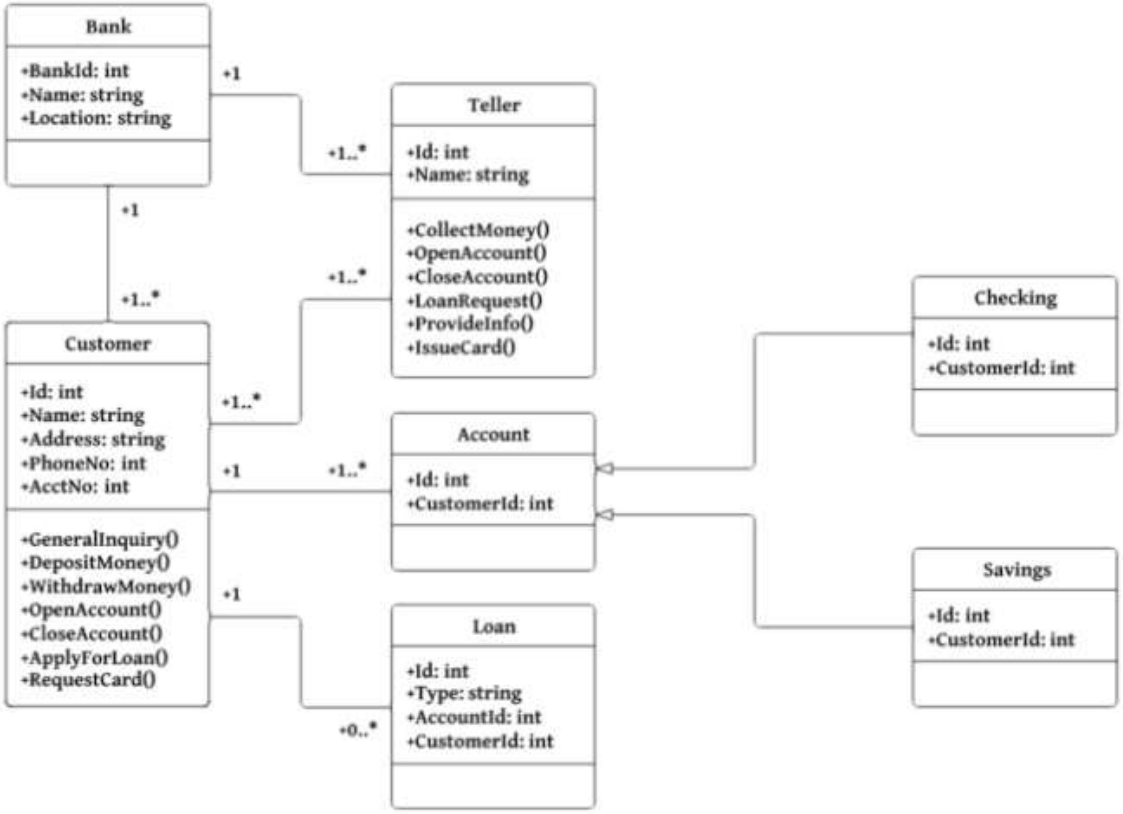
Class Dijagram – primjer 1



Class Dijagram – primjer 2



Class Dijagram – primjer 3

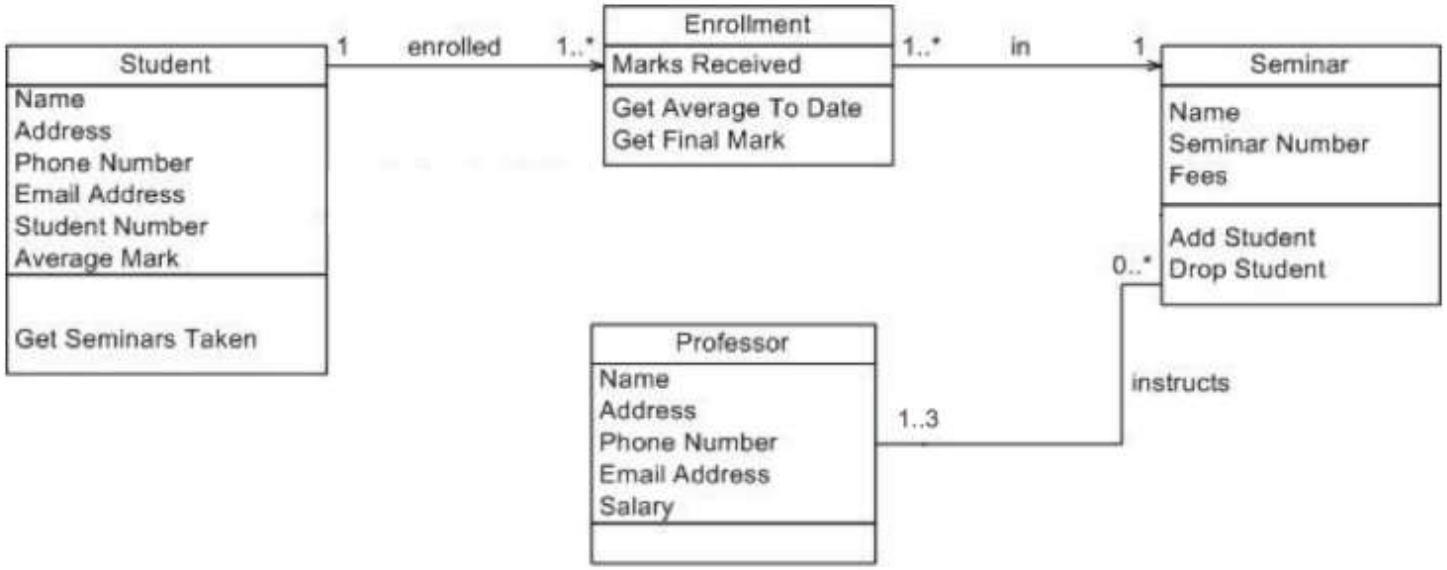


Example of a Class Diagram for a Banking System

Primjer scenarija

Profesori imaju ime, adresu, broj telefona, email adresu i platu. Student ima i ime itd., ali nema platu (izvini). Student, međutim, ima prosječnu ocjenu (od konačnih ocjena njegovih ili njenih seminara). Seminar ima naziv i broj. Kada se student upiše na seminar, ocjene za ovaj upis se evidentiraju i iz upisa se mogu dobiti trenutni prosjek kao i konačna ocjena (ako je ima). Od studenta se može dobiti spisak seminara na koje je upisan. Profesori drže seminare. Svaki seminar ima najmanje jednog, a najviše tri nastavnika. Postoje dvije vrste seminara: bachelor i master. Sa diplomskog seminara studenti se ne mogu povući. Sa master seminara mogu.Nacrtajte dijagram razreda za ovaj univerzitet. Dodajte atribute i metode kada je potrebno. Ne morate uključiti gettere i settere za atribute. Modifikatori vidljivosti (javni, privatni, itd.) nisu potrebni.

Rješenje primjera scenarija



Projektni zadatak 4

- Dizajn arhitekture (Odabir i opis)
- Specifikacija hardvera
- Specifikacija softvera
- Provjera dizajna baze podataka
- Dijagram klasa
- Izvještaj
- Projektni zadatak poslati do 23.12.2024. 23:59 na email: amer.koric@etf.unsa.ba